RETAILMART V3 · SQL

Capstone: RetailMart Analytics Platform

— Siraj Sir

📱 WhatsApp Announcement

Capstone: RetailMart Analytics Platform

This is the day everything you learned becomes ONE real product -- and goes live on the internet with your name on it. You already know how to write the SQL. Today is about understanding the project as a whole, putting it on the web for free, and telling the world you built it.

Part 1: Understand the project -- how every skill from the very first session fits into one analytics platform
Part 2: The dashboard -- a static site that reads your exported JSON
Part 3: Deploy & share -- GitHub Pages, a README, and the LinkedIn post that gets you noticed

By the end you have a shareable link to an enterprise-style analytics dashboard built entirely on SQL -- on your resume, on your GitHub, and on your LinkedIn. This is the single strongest project you can show an interviewer.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Setup: Build the Platform	10 min	Prerequisites & unzip, One command: run_all.sql, Verify 76 views / 13 MVs / 30 indexes, Export JSON + run it locally
Part 1: Understand the Project	Lab 1	How the pieces fit, Full-course -> capstone skill map, The analytics layer & JSON hand-off, The repo, folder by folder
Part 2: The Dashboard	Lab 2	A static site reads JSON, KPI cards & Chart.js, Tabs across the business, Indian currency & light/dark
Part 3: Deploy & Share	Wrap	GitHub Pages in 3 steps, A README others can follow, Post it on LinkedIn, Demoing it in interviews

This Is Where It All Comes Together

For 28 days you learned one skill at a time. The capstone is not a new topic -- it is the moment every one of those skills snaps into a single, deployable product. Before we touch the deployment, let us see exactly where each thing you learned shows up.

DEFINITION

Connect From Day One: Weeks 1-3 -- the foundation under every query

- Filtering & sorting -> the 'Delivered orders only' rule behind every revenue number
- Scalar & conditional logic -> CASE tiers, COALESCE on NULLs, clean labels
- Aggregates & GROUP BY -> every KPI: revenue, order counts, averages
- Joins -> stitch orders + customers + products + regions into one result
- Subqueries & CTEs -> the readable, layered logic inside each view

DEFINITION

Weeks 4-6 -- what turns queries into a product

- Window functions -> RFM scores, running totals, rankings, LAG/LEAD trends
- Query plans & indexing -> why the heavy materialized views are fast enough to power a dashboard
- Pivoting, median & percentiles -> P90/P95 SLAs and cross-tab summaries
- Date/time mastery -> monthly trends, cohorts, time-series buckets
- JSON & semi-structured data -> the get_*_json() hand-off to the dashboard
- Views & materialized views -> the analytics layer itself
- Data quality + cohort/RFM -> trustworthy inputs and the headline analyses

STORY

From Queries to a Product

Until now you wrote queries that answered one question at a time. The capstone is different: you assemble a SYSTEM. Raw tables flow into a tested analytics layer, that layer is exported to JSON, and a dashboard turns the JSON into charts. Same skills you already have -- views, aggregates, window functions -- assembled into something a real company would run.

DEFINITION

Four stages, one direction

- 1. SOURCE: the 16-schema, 55-table RetailMart database
- 2. ANALYTICS: an 'analytics' schema of views + materialized views
- 3. EXPORT: functions turn each analysis into a JSON file
- 4. DASHBOARD: a static site reads the JSON and draws charts

DEFINITION

A real analytics repo has folders, not one big file

- 01_setup: create the analytics schema, metadata tables, indexes
- 02_data_quality: checks the source data is sane before you build on it
- 03_kpi_queries: the views, MVs and get_*_json() per business area
- 05_refresh: refresh all MVs, then export every JSON file
- 06_dashboard: the static site (index.html, dashboard.js, styles.css, data/)
- 07_documentation: the README and notes that explain it to a stranger

REAL WORLD

This Is a Real Job

'Analytics Engineer' and 'BI Developer' do exactly this: model raw data into clean, reusable layers and expose them to dashboards. The folder structure you build here -- setup, data quality, KPI queries, refresh, dashboard -- mirrors a production analytics repository almost exactly. You are not doing a tutorial; you are doing the job.

Setup: Build the Platform in One Command

Before we look inside the analytics layer, get it running on YOUR machine. This whole section takes about ten minutes.

DEFINITION

Prerequisites -- you already have all of these

- PostgreSQL 18 + pgAdmin 4 installed (the setup guides in setup/ cover this)
- The RetailMart V3 database loaded and renamed to accio_retailmart_NN (your batch number)
- The project downloaded: retailmart-analytics-project.zip from the course site
- A terminal (Mac/Linux) or Command Prompt / PowerShell (Windows)

Everything below runs from INSIDE this folder

```
# Unzip the project, then move into it:
cd retailmart_analytics

# Confirm you are in the right place -- you should see these:
#   01_setup/  02_data_quality/  03_kpi_queries/
#   04_alerts/ 05_refresh/  06_dashboard/  run_all.sql
```

The master runner (replace NN with YOUR batch number)

```
psql -d accio_retailmart_NN -f run_all.sql

-- That single command runs all 15 SQL files in the correct order:
--   [1/5] SETUP        -> analytics schema, config, metadata, indexes
--   [2/5] DATA QUALITY -> quality check views
--   [3/5] KPI QUERIES  -> 9 analytics modules (views + MVs)
--   [4/5] ALERTS       -> business alert views
--   [5/5] REFRESH      -> the refresh objects

-- It STOPS at the first error, so you see exactly what failed
-- instead of scrolling through hundreds of lines.
```

REVEAL

What a successful build looks like

- You see the five >>> stage banners scroll past, ending with BUILD COMPLETE
- The final query prints your object counts: 76 views, 13 materialized views, 30 indexes
- If those three numbers match, your analytics layer is fully built
- Nothing in the 55 base RetailMart tables was changed -- you only ADDED an analytics schema

Confirm the objects exist

```
SELECT
  (SELECT COUNT(*) FROM pg_views    WHERE schemaname = 'analytics') AS views,
  (SELECT COUNT(*) FROM pg_matviews WHERE schemaname = 'analytics') AS materialized_views,
  (SELECT COUNT(*) FROM pg_indexes  WHERE schemaname = 'analytics') AS indexes;

-- Expected: 76 | 13 | 30

-- Look around:
\dv analytics.*     -- list the views
\dm analytics.*     -- list the materialized views
```

COMMON MISTAKE

If the Build Fails -- The Four Common Causes

- 'relation sales.orders does not exist' -> wrong database. You pointed at an empty DB, not accio_retailmart_NN.
- 'No such file or directory' on a \i line -> you are not inside the retailmart_analytics folder. cd into it and re-run.
- 'character with byte sequence 0x90 … has no equivalent in encoding UTF8' (Windows) -> you are running an OLD copy. Re-download the project; every file now starts with \encoding UTF8, which fixes this.
- 'permission denied for schema' -> your DB user cannot CREATE. Connect as the owner (usually postgres).

Turn your analytics layer into JSON the dashboard can read

```
cd 05_refresh
chmod +x export_all_json.sh      # Mac/Linux, first time only
./export_all_json.sh

# This writes ~45 JSON files into 06_dashboard/data/
# Use --refresh to rebuild the materialized views first:
./export_all_json.sh --refresh
```

PRO TIP

PRO TIP

On Windows the .sh script needs Git Bash (installed with Git) -- open the folder in Git Bash and run it there. If you would rather not run a script at all, the same JSON can be produced from psql with \copy against each export function; the script simply loops over them for you.

Open the dashboard before you deploy it

```
cd 06_dashboard
python3 -m http.server 8000        # Mac/Linux
python -m http.server 8000         # Windows

# Then open:  http://localhost:8000

# Use a local server, NOT a double-click on index.html --
# opening the file directly blocks fetch() of the JSON files
# (browser CORS rules), so every chart would come up empty.
```

REAL WORLD

You Just Did What a Data Engineer Does

Build the analytics layer, verify object counts, export a serving format, run it locally, then deploy. That loop -- build, verify, serve, ship -- is the actual working rhythm of an analytics engineering job. The only difference in a company is that a scheduler runs the refresh at 2 AM instead of you typing it.

Part 1: Inside the Analytics Layer

You already know how to write views, materialized views and JSON functions. Here we look at how the project organizes them -- not the syntax, but the shape.

STORY

Never Query Raw Tables From a Dashboard

A dashboard should never run a 6-table join itself. Instead, you build that logic ONCE as a view in an analytics schema, materialize the heavy ones for speed, and let the dashboard read simple, pre-shaped results. Clean separation: raw data stays raw, the analytics layer is the contract everything else depends on.

DEFINITION

One schema, three kinds of object, one naming rule

- analytics.vw_* -> plain views: live logic, always fresh, no storage
- analytics.mv_* -> materialized views: cached results that feed the dashboard
- analytics.get_*_json() -> functions that export an analysis as JSON
- The whole layer sits apart from the 16 source schemas -- you never alter raw tables

DEFINITION

Raw -> cleaned -> enriched -> metrics

- Layer 1 (vw_*_clean): join and rename raw tables into tidy columns
- Layer 2 (vw_*_enriched): add business logic -- margins, tiers, flags
- Layer 3 (vw_*_kpi / mv_*): aggregate into the numbers a chart needs
- Each layer is simple; stacked together they produce a complex result

REVEAL

View vs Materialized View (recall the Views session)

- VIEW: always fresh, re-runs every time, no storage. Use for light, building-block logic.
- MATERIALIZED VIEW: stored result, instant reads, must REFRESH. Use for heavy aggregates feeding the dashboard.
- Rule of thumb: dashboard-facing = materialized; building-block = plain view.

You already wrote views like this -- here it is in the project

```
CREATE OR REPLACE VIEW analytics.vw_monthly_sales AS
SELECT
  DATE_TRUNC('month', o.order_date)::date AS month,
  COUNT(*)             AS orders,
  SUM(o.net_total)     AS revenue,
  ROUND(AVG(o.net_total)::numeric, 2) AS avg_order_value
FROM sales.orders o
WHERE o.order_status = 'Delivered'
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY month;
```

PRO TIP

PRO TIP

The headline revenue is SUM(net_total) on Delivered orders. Keep EVERY view consistent with that ONE definition or your numbers will disagree across tabs. Use the real V3 columns -- cust_id (not customer_id), prod_id on order_items, and JOIN core.dim_region for region names.

DEFINITION

02_data_quality runs before you trust a single KPI

- Orphan orders: an order whose customer does not exist (a broken join)
- Negative or NULL revenue that would silently skew every chart
- Duplicate keys, impossible dates, out-of-range values
- If a check fails you fix the source query -- you never build KPIs on dirty data

DEFINITION

How the analytics layer talks to the dashboard (recall the JSON session)

- Each analysis gets one analytics.get_*_json() function
- The function reads its materialized view and returns dashboard-ready JSON
- That JSON is written to a file in 06_dashboard/data/
- The dashboard fetches the file -- it never connects to the database

An export function (recall the JSON session) -- analysis in, JSON out

```
CREATE OR REPLACE FUNCTION analytics.get_sales_summary_json()
RETURNS JSON LANGUAGE plpgsql STABLE AS $$
BEGIN
  RETURN json_build_object(
    'monthly', (
      SELECT json_agg(json_build_object(
        'month',   to_char(month, 'Mon YYYY'),
        'orders',  orders,
        'revenue', revenue
      ) ORDER BY month)
      FROM analytics.mv_monthly_sales
    )
  );
END; $$;
```

DEFINITION

One script regenerates the whole dashboard

- Step 1: REFRESH every materialized view (base MVs before MVs that read them)
- Step 2: loop over every get_*_json() function
- Step 3: write each result to 06_dashboard/data/<name>.json (~40 files)
- export_all_json.sh does all of this in one run -- refresh data, re-run it

COMMON MISTAKE

Capstone Pitfalls (Hard-Won)

- Wrong column names: it is cust_id, prod_id, net_total -- not customer_id/product_id.
- Region lives in core.dim_region -- JOIN it; stores has region_id, not a name.
- PERCENTILE_CONT / AVG return double -> cast ::numeric before ROUND.
- REFRESH CONCURRENTLY fails without a UNIQUE index on the MV.
- Refresh order matters: refresh base MVs before MVs that read them.

Part 2: The Dashboard

A static site that reads your exported JSON -- no server, no database password, no backend to host.

STORY

No Server, No Database Password, Just Files

The dashboard is a plain HTML page. It does not connect to PostgreSQL -- it reads the JSON files you exported. That means it can be hosted anywhere for free, loads instantly, and never exposes your database. This is the standard way analysts ship a portfolio dashboard.

DEFINITION

Static site = HTML + JS + JSON

- index.html lays out KPI cards, chart canvases and tabs
- dashboard.js fetches the JSON files and draws charts with Chart.js
- styles.css gives it the light/dark enterprise look
- data/*.json is the only thing that changes when data refreshes

dashboard.js -- the core loop

```
// 1. Load the exported data (no backend needed)
const res  = await fetch('./data/sales_summary.json');
const data = await res.json();

// 2. Hand it to Chart.js
new Chart(document.getElementById('chartRevenue'), {
  type: 'line',
  data: {
    labels:   data.monthly.map(r => r.month),
    datasets: [{ label: 'Revenue', data: data.monthly.map(r => r.revenue) }]
  }
});
```

A KPI card is just HTML + one number

```
<!-- index.html -->
<div class="kpi-card" id="kpiRevenue">
  <div class="kpi-label">Total Revenue</div>
  <div class="kpi-value">--</div>
</div>

// dashboard.js -- fill it after loading JSON
document.querySelector('#kpiRevenue .kpi-value')
        .textContent = formatINR(data.totalRevenue);
```

DEFINITION

One tab per area of the business

- Executive: headline revenue, orders, customers, AOV + trends
- Sales, Customers, Products, Stores: deep-dive KPIs and charts
- Operations, Marketing, Finance & HR, Audit, Supply Chain
- A Schema tab proving the analytics layer uses all 55 tables

PRO TIP

PRO TIP

Format money the Indian way -- crores and lakhs (Rs 676.95 Cr, Rs 84.30 L), not millions. Small touches -- correct currency, a light/dark toggle, consistent revenue definitions -- are what make a student project look professional instead of like a tutorial.

Part 3: Deploy & Show the World

Free hosting on GitHub Pages, a README anyone can follow, and the LinkedIn post that turns your project into opportunities.

STORY

A Project Nobody Can See Does Not Count

The difference between 'I built a dashboard' and a live URL an interviewer can click is enormous. GitHub Pages hosts your static dashboard for free, straight from your repository, in about three minutes. Then a short README turns your repo into something a stranger can understand and run.

DEFINITION

Free static hosting, built into your repo

- GitHub serves the files in your repo as a public website -- no server to rent
- Perfect for a static dashboard: just HTML, JS and JSON files
- You get a permanent URL: https://<username>.github.io/<repo>/
- Every push updates the live site automatically

Free hosting in three steps

```
# 1. Push your project (with the dashboard) to a PUBLIC repo
git add .
git commit -m 'Add RetailMart analytics dashboard'
git push origin main

# 2. On GitHub: repo -> Settings -> Pages
#    Source: 'Deploy from a branch'
#    Branch: main   Folder: /root  (or /docs if you move the site there)

# 3. Wait for the green check, then open your live URL:
#    https://<username>.github.io/<repo>/06_dashboard/
```

PRO TIP

PRO TIP

Use RELATIVE paths in the dashboard -- fetch('./data/sales_summary.json'), not a leading slash -- or the data will 404 once it is under /<repo>/. Commit the data/ folder (the JSON files must be in the repo). The repo must be public for free Pages. First deploy can take a minute -- refresh the Pages settings until the URL appears.

COMMON MISTAKE

Why the Live Page Is Blank (and the fix)

- 404 on the page -> wrong branch/folder in Pages settings, or site is in a subfolder
- Page loads but charts are empty -> data/*.json not committed, or absolute /data path
- Works locally, breaks live -> filename case mismatch (Pages is case-sensitive)
- Nothing happens for a minute -> Pages is still building; give it time, then hard-refresh

DEFINITION

Five things every good README has

- One-line description + the live link (and a screenshot) at the very top
- The architecture in 3 lines: source -> analytics layer -> JSON -> dashboard
- How to run it: setup SQL, refresh/export script, open the dashboard
- What it demonstrates: views, MVs, JSON export, charts
- Tech used: PostgreSQL, SQL, Chart.js, GitHub Pages

STORY

Shipped But Silent = Invisible

A live project that nobody knows about helps nobody. Recruiters, mentors and peers find you through your posts, not your private repos. A single well-made LinkedIn post about a project you can actually demo is one of the highest-return things you will do this whole course -- it turns weeks of work into reach.

DEFINITION

Hook, proof, link, and a way to engage

- A one-line hook: what you shipped, not 'I completed a course'
- 3-4 bullets on what it is and the skills behind it
- The LIVE link + the GitHub repo link
- Media: a screenshot, or better, a 30-60s screen recording (video gets more reach)
- Tag AccioJob and your mentor; thank them -- it widens the audience
- 3-5 relevant hashtags, no more

DEFINITION

Pick Your Angle -- same project, four very different posts (choose one that is YOU)

- The Transformation: 'From my first SELECT to a deployed platform in a few weeks' -- the journey
- The Builder: what was HARD, how you solved it, what you would add next -- honest and human
- The Insight: lead with one surprising number you found ('I analysed Rs 676 Cr of orders and...')
- The Teacher: explain ONE concept you now love -- materialized views, JSON export -- to your network

Make It Yours -- complete these in your own words (no two will match)

```
Hook   -> The one thing I'm proud I can now do: ____
What   -> In one sentence, my project is: ____
Proof  -> A number or feature unique to MY build: ____
          (tables analysed, charts shipped, a metric I surfaced...)
Skills -> The 3 skills I leaned on most: ____, ____, ____
Hard   -> The trickiest part and how I cracked it: ____
Next   -> One thing I'd add with more time: ____
CTA    -> What I want from readers (feedback / connections / roles): ____
Links  -> Live: ____    Code: ____
Thanks -> Siraj Sir + AccioJob
Tags   -> pick 3-5: #SQL #PostgreSQL #DataAnalytics #AnalyticsEngineering ...
```

Let AI Draft It -- paste this in, fill the brackets, then make it sound like you

```
Write a short, authentic LinkedIn post (120-180 words) about a data
project I built. Avoid hype and buzzwords. First person, confident but
humble. End with 3-5 hashtags.

Details:
- Project: a SQL analytics platform on PostgreSQL with a live dashboard
- What makes mine unique: <your angle / a number / a feature>
- Skills used: <e.g. views, materialized views, window functions, JSON>
- Hardest part: <what challenged you and how you solved it>
- Live link: <pages link>   Code: <github link>
- Thank: Siraj Sir and AccioJob
```

REVEAL

One Example (the 'Transformation' angle) -- write yours, don't copy mine

"A few weeks ago I had never written a SQL query. Today I deployed an analytics platform that turns a 55-table retail database into a live dashboard."

"I built an analytics layer of views and materialized views, exported it to JSON, and shipped a Chart.js dashboard on GitHub Pages -- no backend, no exposed database."

"The hardest part was keeping every revenue number consistent across tabs. Next I want to add a cohort-retention view."

"Live: <link> Code: <link> Thanks Siraj Sir & AccioJob. #SQL #PostgreSQL #DataAnalytics"

PRO TIP

PRO TIP

Make the post work harder: upload the screenshot or video natively (LinkedIn favors media over bare links). Pin the post to the Featured section of your profile. Reply to every comment in the first hour -- early engagement decides reach. If you want maximum reach, put the live link in the FIRST comment rather than the post body.

DEFINITION

One project, many surfaces

- Resume: a bullet with the live link -- 'Built & deployed a SQL analytics platform (live link)'
- GitHub profile README: feature the repo with a screenshot
- Portfolio site: embed the dashboard or link it
- Interviews: have the URL ready to open and walk through

REAL WORLD

Why This Wins Interviews

A resume bullet says 'I know SQL'. A live link to an analytics dashboard -- backed by views, materialized views and JSON exports you can explain -- PROVES it. Interviewers remember the candidate who shipped a real product over the one who only solved puzzles. Be ready to walk through the architecture in two minutes.

INTERVIEW

Q: Walk me through your analytics project.

A: 'Raw RetailMart data lives in 16 schemas. I built an analytics schema of views for clean logic and materialized views for the heavy, dashboard-facing aggregates. Export functions turn each one into JSON, a refresh script regenerates everything, and a static Chart.js dashboard reads the JSON -- so it deploys free on GitHub Pages with no backend and no exposed database.' Then show the live link.

CHALLENGE

Build, Deploy & Share Your Analytics Platform

GOAL: Produce a working analytics layer and a deployed dashboard for RetailMart, in your own database, and share it publicly.

REQUIRED: (1) an analytics schema with at least 6 views and 3 materialized views; (2) get_*_json() functions exporting your KPIs; (3) a dashboard with KPI cards + at least 6 charts across 3+ business areas; (4) a public GitHub Pages link; (5) a README; (6) a LinkedIn post with the live link.

STRETCH: Add a light/dark toggle, Indian-currency formatting, a data-quality page, a Schema tab listing the tables you used, and a 30-60s demo video in your post.

KEY TAKEAWAYS

It all connects: The capstone is not a new topic -- it is every skill from the whole course assembled into one product.
Model, do not query: Build an analytics schema of views + MVs. The dashboard reads the layer, never the raw tables.
JSON is the hand-off: get_*_json() functions turn SQL results into files a static site can read -- no backend required.
Ship it publicly: GitHub Pages gives you a free live URL; a clear README lets a stranger run it.
Then share it: Post it on LinkedIn with the live link. A project you can demo -- and that people can find -- beats any resume line.

Congratulations

You started by writing your first SELECT. You are finishing by shipping an enterprise-style analytics platform built entirely on SQL: a clean analytics layer, JSON exports, and a live dashboard. Deploy it, put the link on your resume, post it on LinkedIn, and walk an interviewer through the architecture. This is the project that gets you hired.